

MID-STATE TECHNICAL COLLEGE • IT PROGRAMS

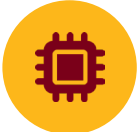
System Modification and Maintenance

Unit 5 • Virtualization and SDDC (10-150-130)



Learning Outcomes

By the end of this unit, you will be able to:



Modify VM and container resources

Change CPU, memory, and limits with the web UI, qm, and pct — and verify the result in the guest.



Determine what can change live

Enable hot-plug, predict which changes apply immediately, and recognize pending changes.



Move storage and reconfigure networking

Migrate disks from NFS to local ZFS, add mount points and NICs, and clean up what is left behind.



Diagnose and manage contention

Use CPU weights and limits, ballooning, and I/O limits to keep neighbors from starving each other.



Plan monitoring and maintenance

Baseline performance, schedule preventive work, and document every change.

LESSON 1

Modifying VMs and Containers



Why Modify After Deployment?

- Workloads change after go-live — more users, a new service, a database that keeps growing.
- **Right-sizing** corrects the initial guess in both directions: starved guests run slowly, oversized guests waste capacity their neighbors need.
- In a VM, a resource change is just an edit to its configuration file in `/etc/pve/qemu-server/` — no hardware to touch.
- The skill is knowing **when** a change takes effect and **how** to back it out.



Start small, grow on evidence

Adding resources to a guest is usually easy and often live. Taking them away is usually not.

Deploy lean, watch the graphs, and grow when the data says so.

Applied Now or Pending?

Every change is either applied to the running guest immediately or queued as a pending change:

```
qm config 100      # configuration currently in effect
qm pending 100     # current values next to any pending ones
```

- The web UI flags pending values in color on the VM's **Hardware** tab until they are applied.
- Pending changes apply when Proxmox **stops and starts** the VM — a reboot from inside the guest keeps the same QEMU process and does not apply them.
- Containers apply most resource changes (cores, memory, swap) immediately through cgroups.

LIVE DEMO

Change a VM's CPU type, show the pending entry, then stop/start and confirm it cleared.

Scaling CPU: 1 → 2 Cores

- Total vCPUs = **sockets × cores**. For performance the split rarely matters; for per-socket licensing it does.
- Without CPU hot-plug, a core change is **pending** until the next full stop/start.
- Verify from inside the guest with `nproc` or `lscpu` — the config is not proof.

```
qm set 100 --cores 2
```

Overcommit is normal

Proxmox lets the total vCPUs of all guests exceed the physical cores — the host schedules QEMU threads like any other process.

It will not start a single VM with more vCPUs than the host has.

LIVE DEMO

Scale a Linux VM from 1 to 2 cores, stop/start it, and confirm with `lscpu`.

Expanding Memory: 2 GB → 4 GB

Memory settings live on the VM's Hardware tab; values are in MiB.

Setting	Config key	What it controls
Memory	memory	Maximum RAM the guest can use — 4096 for 4 GB
Minimum memory	balloon	RAM always guaranteed; lower than Memory enables automatic allocation
Ballooning device	balloon: 0	Disables the balloon device (debugging only — leave it on)
Shares	shares	Relative priority for spare host RAM (default 1000)

Example: `qm set 100 --memory 4096 --balloon 2048` — 2 GB guaranteed, up to 4 GB when the host has room.

LIVE DEMO

Raise a VM from 2048 to 4096 MiB and check `free -m` in the guest before and after.

VMs vs. Containers: Changing Resources

Virtual machine (qm)

- `qm set 100 --cores 2 --memory 4096`
- Guest kernel must detect new hardware — live only with hot-plug
- Otherwise pending until a full stop/start

Container (pct)

- `pct set 200 --cores 2 --memory 1024`
- `--swap`, `--cpulimit`, `--cpuunits` apply the same way
- Applied immediately — no restart, no hot-plug setting

Why containers change instantly: a container shares the host kernel, so a new limit is just a new cgroup value — there is no virtual hardware for a guest to discover.

LIVE DEMO

Raise a running container's memory with `pct set` and watch `free -m` update in its console.

Snapshot Before You Change

- Take a **snapshot** before any change you might need to undo: `qm snapshot 100 pre-change`
- Roll back with `qm rollback 100 pre-change`; containers use `pct snapshot` and `pct rollback`.
- Include RAM (`--vmstate 1`) when you need to return to the exact running state.
- Delete the snapshot once the change is verified — old snapshots grow and slow writes.



SECURITY FOCUS

A snapshot lives on the same storage as the disk it protects. It undoes a bad change; it does not survive a failed pool.

Why it matters: Treating snapshots as backups is how a single storage failure becomes permanent data loss.

LESSON 2

Hot-Plug and Its Limits



Enabling Hot-Plug

- Set per VM under **Options → Hotplug** — each device class is enabled separately.
- The default is `disk, network, usb`; **CPU** and **memory** hot-plug are off.
- CPU and memory hot-plug also require **NUMA** enabled on the Processor settings.
- Changing the hot-plug setting is itself applied at the next stop/start — build it into templates.



Prepare before you need it

Hot-plug you enable during an incident does nothing until the VM restarts — which is the downtime you were trying to avoid.

Hot-Plug Support Matrix

What Proxmox VE can change on a running VM, and what the guest must support:

Component	Add while running	Remove while running	Requirement
vCPU	Yes — <code>cpu</code> + NUMA	A request; the guest may refuse	Linux guest; kernel 4.7+ recommended
Memory	Yes — <code>memory</code> + NUMA	Unreliable — plan a restart	Guest must bring new memory online
Network interface	Yes (default)	Yes (default)	Guest driver (VirtIO on Windows)
Disk	Yes (default)	Yes — unmount in guest first	VirtIO SCSI or VirtIO Block bus
Firmware / machine / CPU type	No	No	Pending until a full stop/start

Hot-unplug of CPU and memory depends on guest cooperation through ACPI — treat it as best effort, never as a plan.

vCPU Hot-Plug with vcpus

Sockets × cores sets the maximum; vcpus sets how many are plugged in at start:

```
qm set 100 --numa 1 --hotplug disk,network,usb,cpu,memory  
qm set 100 --sockets 1 --cores 4 --vcpus 2
```

- Raise `vcpus` later (up to 4 here) and the new CPUs are hot-added to the running guest.
- Linux may leave new CPUs **offline** — a udev rule in the guest brings them up automatically.
- Proxmox recommends `cpulimit` for most cases; hot-plug is for when you truly cannot restart.

Guest rule, saved as `/etc/udev/rules.d/80-hotplug-cpu.rules`:

```
SUBSYSTEM=="cpu", ACTION=="add", TEST=="online", ATTR{online}=="0", ATTR{online}="1"
```

Guest and Application Readiness

Guest operating system

- Modern Linux handles CPU, memory, NIC, and disk hot-plug well
- Windows needs the VirtIO drivers for NIC, disk, and balloon devices
- Proxmox does not recommend the Windows balloon driver on critical systems

Applications

- Fixed-size settings — database buffer pools, JVM heap — ignore new RAM until reconfigured
- Thread pools sized at startup may not use new cores
- Some licenses count cores or sockets — check before adding

Test the full stack: make the change in a staging VM first, then confirm the application actually uses the new resource.

When a Restart Is Still Required

- Changing **firmware** (SeaBIOS ↔ OVMF), **machine type**, or **CPU type**
- Enabling **NUMA** or changing the **hot-plug** setting itself
- Reducing **memory** or **cores** on a guest that will not release them
- Adding a disk or NIC on a bus or device class not enabled for hot-plug



Plan the window

Batch restart-required changes into one maintenance window.

Announce it, snapshot first, and verify each change after the start.

LESSON 3

Storage and Network Changes



Moving a Disk Between Storage

- Select the disk, then **Disk Action → Move Storage**, and pick the target.
- A running VM keeps working — Proxmox mirrors the disk live, then switches over.
- **Delete source** removes the old copy; without it the original remains as an `unusedN` disk.

```
qm disk move 100 scsi0 local-zfs --delete 1
```

Format changes too

`qcow2` on NFS becomes a `raw` zvol on ZFS — block storage has no `qcow2`.

Snapshots do not travel with the disk.

LIVE DEMO

Move a running VM's disk from VMLab-NFS to local-zfs and follow the task log.

NFS vs. Local ZFS

The move is a trade-off, not an upgrade:

	NFS (VMLab-NFS)	Local ZFS (local-zfs)
Location	Shared over the network	Disks inside this host
Disk format	File — <code>qcow2</code> or <code>raw</code>	Block — <code>raw</code> zvol
Snapshots	<code>qcow2</code> internal snapshots	ZFS snapshots (fast, efficient)
Migration	Disk stays put — only RAM moves	Disk must be copied to the target
Performance	Bounded by network and NAS	Local latency, ARC caching
Depends on	Network and storage server up	This host's disks only

Proxmox documents raw images as up to about 10% faster than qcow2 — the format alone does not explain the whole difference.

Unused Disks: Find Them, Then Remove Them

A detached or un-deleted disk stays in the VM's configuration as unusedN:

```
scsi0: local-zfs:vm-100-disk-0,ioread=1,size=32G
unused0: VMLab-NFS:100/vm-100-disk-0.qcow2
```

- Unused disks still consume space and still appear in storage content.
- Remove one by selecting it on the Hardware tab and choosing **Remove**.
- Proxmox refuses to delete a moved disk's source while snapshots reference it — clean up snapshots first.



Detach ≠ delete

Detach first and confirm the VM boots.
Remove the unused disk only after you are sure it is not needed.

Container Mount Points and NFS

The host mounts the share; the container receives a bind mount of one directory:

```
pct set 200 -mp0 /mnt/pve/VMLab-NFS/shared,mp=/mnt/shared
```

- mp0–mp255 attach host paths or new storage volumes (local-zfs:8,mp=/data).
- Proxmox mounts NFS storage on the host under /mnt/pve/<storage>.
- In an **unprivileged** container, UIDs are shifted by 100000 — files may appear owned by *nobody*.



SECURITY FOCUS

A bind mount exposes a host path inside the container. Mount the narrowest directory, read-only where you can.

Why it matters: Mounting a broad host path hands the container data it was never meant to see.

Reconfiguring Network Interfaces

Network devices are hot-pluggable by default — add, move, or tag them on a running guest:

Change	Command
Add a second NIC	<code>qm set 100 --net1 virtio,bridge=vmbr1</code>
Move to another bridge	<code>qm set 100 --net0 virtio=BC:24:11:AA:BB:CC,bridge=vmbr1</code>
Tag a VLAN	<code>qm set 100 --net0 virtio=BC:24:11:AA:BB:CC,bridge=vmbr0,tag=20</code>
Container on DHCP	<code>pct set 200 --net0 name=eth0,bridge=vmbr0,ip=dhcp</code>

Keep the MAC: omit it when redefining net0 from the CLI and Proxmox generates a new one — DHCP reservations and interface names can break.

Boot Order and Automatic Startup

Start at boot

- Options → **Start at boot** — or `--onboot 1`
- Without it, guests stay off after a host reboot
- Containers use the same option through `pct`

Startup order

- **order** — lower starts first, shuts down last
- **up** — seconds to wait before starting the next guest
- **down** — shutdown timeout before a forced stop

```
qm set 100 --onboot 1 --startup order=1,up=60
```

LIVE DEMO

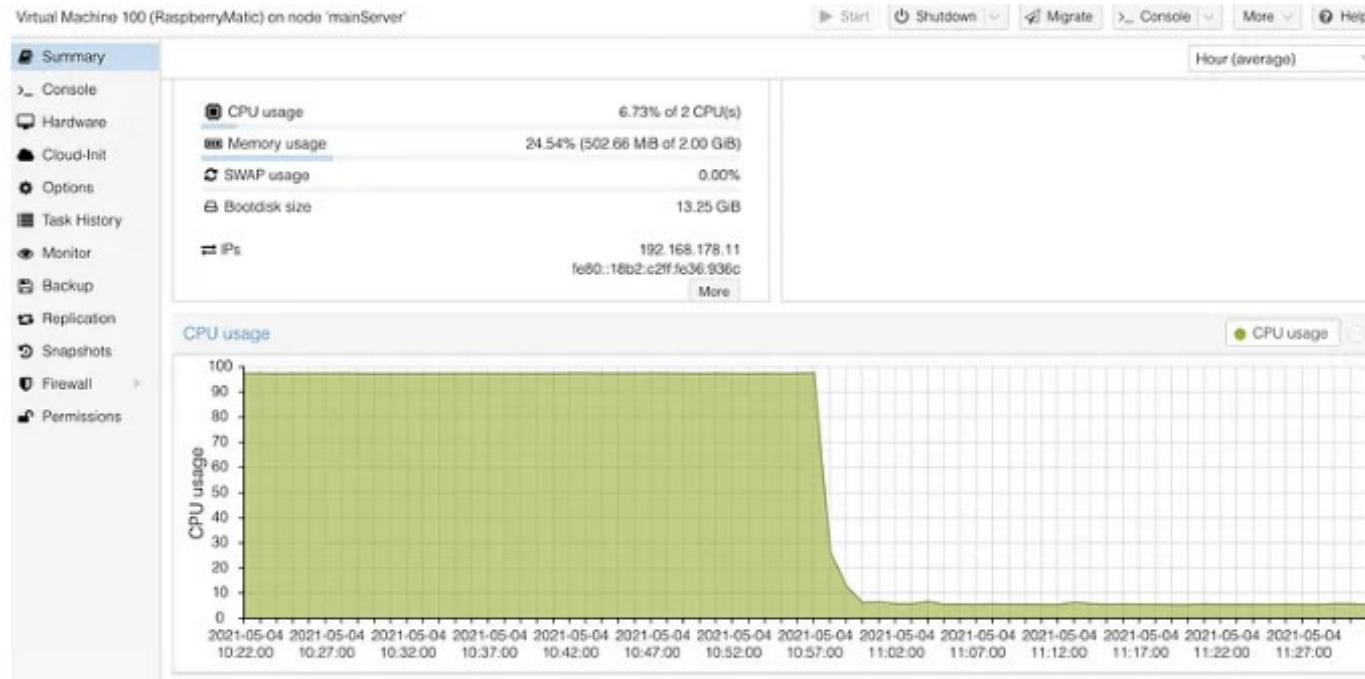
Set a DNS VM to start first with a 60-second delay, then review the Options tab.

LESSON 4

Managing Contention



Recognizing Contention



Signs of contention

High **st** (steal) in **top** inside a Linux guest
Rising **IO delay** on the node summary
Guest swapping while its balloon shrinks

Contention is a host-level problem: a guest graph alone cannot tell busy from starved.

CPU Controls

Four levers, from most to least commonly used:

Control	Config key	Effect	Use when
Cores	cores	Number of vCPUs the guest sees	The workload is truly parallel
CPU weight	cpuunits	Relative share under contention (default 100)	One guest must win a busy host
CPU limit	cpulimit	Hard cap on host CPU time — 2.0 = 200%	Many vCPUs, but a capped total
Affinity	affinity	Pins vCPUs to host cores, e.g. 0-1	Licensing or latency — rarely

Example: `qm set 100 --cpuunits 200` gives VM 100 twice the CPU time of a default guest when they compete.

Memory Ballooning and Shares

- The **balloon driver** in the guest hands RAM back to the host or takes more, between minimum and maximum.
- The host reclaims memory when its usage passes the **ballooning target** — 80% by default.
- **Shares** split the spare RAM among ballooning guests in proportion to their values.
- Every Linux release since 2010 includes the driver; Windows needs VirtIO drivers.

Worked example

32 GB host, 80% target, 16 GB in use → about 9 GB spare.

Database VM: `shares 3000`. Three web VMs: `1000` each.

Database gets $9 \times 3000 \div 6000 = 4.5$ GB extra; each web VM gets **1.5 GB**.

Storage and Network Throttling

Per-disk and per-NIC limits keep one noisy guest from starving its neighbors:

Limit	Config option	Notes
Disk bandwidth	<code>mbps_rd / mbps_wr</code>	MB/s per disk; can be changed on a running VM
Disk IOPS	<code>iops_rd / iops_wr</code>	Operations per second — protects shared arrays
Burst allowance	<code>mbps_wr_max, iops_wr_max</code>	Short bursts above the sustained limit
NIC rate	<code>rate</code>	MB/s on a netX device — megabytes, not megabits

```
qm set 100 --scsi0 local-zfs:vm-100-disk-0,mbps_wr=100,iops_wr=500
```

NUMA and CPU Locality

- In multi-socket servers, each socket has its own local memory — **NUMA** (Non-Uniform Memory Access).
- Reaching the other socket's memory is slower; large VMs should stay within one node when possible.
- Enable NUMA on the VM and match its sockets to the host's node count.

```
numactl --hardware | grep available  
lscpu | grep -i numa
```



In our lab

The HP DL20 Gen10 hosts are single-socket — one NUMA node.

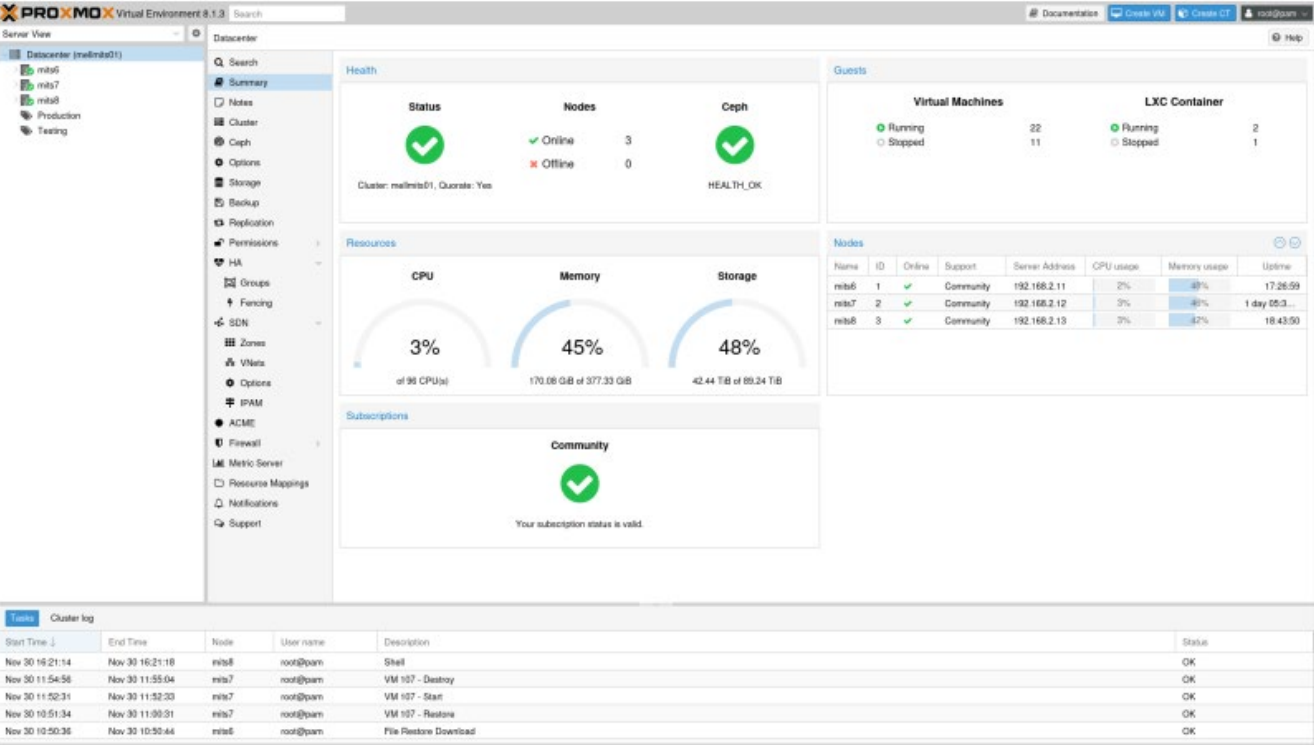
You will see the concept, not the penalty.
Expect it on two-socket production servers.

LESSON 5

Monitoring and Maintenance



Reading the Summary Graphs



Baseline first

Record normal CPU, memory, I/O, and network for each guest before changing anything.

Views run from hour to year; switch from average to **maximum** to see spikes.

LIVE DEMO

Drill from Datacenter to node to VM, and compare average vs. maximum on the week view.

Command-Line Health Checks

Quick answers from the host shell:

Command	What it tells you
<code>pvesh get /cluster/resources</code>	Every guest, node, and storage with current usage
<code>qm status 100 --verbose</code>	Live CPU, memory, balloon, and disk I/O counters
<code>pvesm status</code>	Capacity and usage for each configured storage
<code>zpool status / zpool list</code>	Pool health, errors, and fill level
<code>top (inside a guest)</code>	Steal time — the guest waiting on the hypervisor

For history beyond the built-in graphs, send metrics to an external InfluxDB or Graphite server under Datacenter → Metric Server.

Preventive Maintenance

Weekly

- Apply updates in a window — `apt update && apt dist-upgrade`
- Check pool health — `zpool status`
- Delete verified snapshots and unused disks

Monthly / quarterly

- Test-restore a `vzdump` backup, not just the job status
- Review graphs against the baseline; right-size guests
- Check hardware health and firmware in iLO5

Updates need a plan: a kernel update takes effect only after a host reboot — migrate or shut down guests first, and snapshot anything you might roll back.

Change Management

- **Plan** — what changes, why, the impact, and the rollback.
- **Approve and schedule** — agree on a window with the service owner.
- **Execute** — snapshot, change, verify in the guest and the application.
- **Document** — record it in the guest's **Notes** panel and the change log; remove the snapshot.



SECURITY FOCUS

The Proxmox task log records who ran every change — as long as each admin signs in with a personal account, never a shared root login.

Why it matters: An undocumented change on a shared account cannot be traced, reviewed, or safely undone.

Troubleshooting Failed Modifications

Symptom	Likely cause	Fix
New RAM not visible in guest	Memory hot-plug or NUMA off; guest not onlining memory	Enable both, then stop/start the VM
New vCPUs show offline	No CPU udev rule in the guest	Add the rule, or write 1 to <code>cpuN/online</code>
Change stays highlighted	Pending — cannot apply live	Stop and start the VM from Proxmox
Disk move target not listed	Storage lacks the Disk image content type	Enable the content type; check <code>pvesm status</code>
Container fails after mp0	Host path missing or NFS not mounted	Verify the path and storage on the host

Security Across This Unit

Every modification is also a security decision:



SECURITY FOCUS

Rollback is not recovery — snapshots share storage with the disk; tested backups survive its failure.

Least exposure — bind-mount the narrowest host path into containers, read-only where possible.

Resource limits are availability controls — one runaway guest should not take down its neighbors.

Accountability — personal accounts plus documented changes make every modification traceable.

Why it matters: Most virtualization outages are self-inflicted changes. Discipline around change is the cheapest security control you have.

Hands-On Lab: PVE8_SM_Lab_06

NETLAB · Modifying Virtual Machines and Containers · details in Blackboard Ultra

- Scale a VM from 1 to 2 cores and from 2 GB to 4 GB of memory.
- Move a virtual disk from NFS to local ZFS and remove the unused disk.
- Create an NFS mount point for a container.
- Add and reassign network interfaces; set boot order and automatic startup.



Graded on the live configuration and in-guest verification — a working change, not a screenshot.

Key Takeaways



Snapshot, change, verify in the guest, document — then delete the snapshot.



Hot-plug works only if it was enabled before you needed it; some changes always need a restart.



Moving storage trades mobility for performance — and leaves unused disks to clean up.



Weights, limits, ballooning, and throttles decide who wins when resources run short.



Baselines, scheduled maintenance, and change records keep small problems small.

MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Questions?

Unit 5 · System Modification and Maintenance